

J Primer

作者: Eric Iverson 翻译: Taigua

目录

1 从这里开始	3
2 为什么学J?	4
3 本书的目标	6
4 需要的背景知识	7
5 如何使用本书	8
6 开始	9
7 实验	10
8 标准配置	12
9 术语	13
10 字母表	14
11 单词	16
12 句子	17
13 动词	18
14 名词	19
15 数字	20

16 负数	22
17 原语	24
18 名字	25
19 注释	27
20 错误	28
21 两可性	29
22 双子	30
23 单子	31
24 NuVoc - 新词汇表	32
25 第一个检查点	35
26 数值常量	36
27 字符串	37
28 单词构成	39
29 空格	42
30 优先级	45
31 括号	46
32 求值顺序	47

第 1 章

从这里开始

J是一种通用的高级编程语言。如果您是J的新手，并且想成为一名J程序员，那么这是一个很好的起点。即使您有相当丰富的编程经验，J也有很多独特之处，在深入研究之前，至少浏览一下这些内容是值得的。

第 2 章

为什么学J?

J是一种非常丰富的语言。你可能学习和使用它很多年，但仍然认为自己是初学者。这与Basic或Java等更简单的语言形成鲜明对比，在这些语言中，数月的努力学习和使用将使您成为专家。成为一名专家J程序员所需要的努力与成为一名专家c++程序员所需要的努力更接近。

好消息是，J的本质是如此简单和一致，你可以很快地学会解决真正有趣的问题。

学习足够的Basic或Java来解决琐碎的问题更容易，但学习足够的J来解决更有趣和更具挑战性的问题更容易。一旦你掌握了这种水平的技能，你就不会走到路的尽头，而是可以继续提高，使自己成为一个更好、更强大的程序员。

J尤其擅长对阵列数据进行数学、统计和逻辑方面的分析。它是一个强大的工具，可以为老问题构建新的更好的解决方案，甚至可以更好地找到尚未完全理解的问题的解决方案。

作为一种通用编程语言，J系统还提供：

- 两个集成的开发环境。JHS和JQt
- 标准库、实用程序和包
- 为应用程序构建事件驱动图形用户界面（JQt）
- 几种与其他编程语言和应用程序接口的方法
- 快速应用程序原型和开发
- 免版税发布应用程序的运行时版本

如果你对具有挑战性的数据处理问题的编程解决方案感兴趣，那么在学习J
上投入的时间将是值得的。

第 3 章

本书的目标

[NuVoc](#)是J语言的权威和明确的规范。[J字典](#)也可以用来学习J，但正因为它简洁、完整且严谨地涵盖了这门语言，并且更侧重于复杂之处而非寻常之处，它确实把我们中的一些人吓跑了。

本书为初学者提供了一个更亲切、更温和的起点。它带着你沿着一条路径，以轻松的步骤逐步前进，直到你能够用J语言编写应用程序。在这个过程中，你会通过简化和具体的情境来接触J的所有关键概念。这本手册的目的是让你快速上手，达到能够使用NuVoc和J字典的程度，并且用起来会让你纳闷：当初为什么还要为这些简单的东西费心呢？

你应该能够相当快地完成入门，最后将成为入门级的J程序员。这样，你将比最有经验的Basic或Java程序员拥有更强大的编程能力。

第 4 章

需要的背景知识

本书假设你熟悉另一种编程语言，如Basic、Java或C。然而，这不是先决条件，即使J是你的第一门计算机语言，应该也不会有什么特别的问题（实际上，更应祝贺你！）。

大多数事情都可以在J中完成，就像在其他语言中完成一样，并且通常一个主题的引入方式和在其他语言中引入它一样，这使得在J中更容易理解它是如何工作的。在某些情况下，有一种更好的J方法来解决这个问题，也会介绍这种方法。

第 5 章

如何使用本书

本书是一系列小的，易消化的部分，适合从头到尾按顺序阅读。一个章节通常需要你读过大部分甚至所有之前的章节。跳来跳去毫无意义，而且可能令人沮丧。

本书是自包含的，可以在不访问J系统的情况下读取。特别是，与J系统交互的示例显示了输入的内容以及系统如何响应。然而，它的目的是在访问系统和尽可能多地使用系统的情况下读取。强烈建议你最后输入所有示例，并尽可能多地把玩它们的变体。

有时章节使用的术语和概念直到后面才定义。这就要求你在第一次阅读时有一个初步的理解并继续，在第二次阅读时它们将变得更加具体。

本书最好读三遍：

- 浏览整本书。可以尝试一些例子，但最好持续前进，了解全局。
- 第二次仔细阅读并尝试所有的例子。
- 第三次尝试自己的例子来澄清你的理解，并提升你实际使用系统机制的舒适度（而不仅仅是阅读它和遵循说明）。

第 6 章

开始

本章假设你在 [JQt IDE](#)（交互式开发环境）中使用 [最新版本的J](#)。

Term窗口是一个执行窗口。你在Term窗口中键入J语句，当按Enter键时，J将执行它们并显示结果。

在Term窗口中键入以下行并按Enter键。

```
2 + 3
```

语句被执行并显示结果。输入以下行并按Enter键。

```
5 - 3
```

你的会话应该看起来像这样：

```
2 + 3
```

```
5
```

```
5 - 3
```

```
2
```

你输入的行缩进三个空格，J的答案从行首开始。

第 7 章

实验

我们鼓励你去尝试。尝试用不同的数字输入类似的行。很明显，+是加法，-是减法。输入一些用*表示乘，用%表示除的行。通过使用%，你将很快看到结果可以是2.5这样的数字，并且它们也可以用作参数。

在你有更多的经验之前，你有时可能会对你所观察到的东西感到惊讶甚至不安。一步一步来。尝试一些你确信你已经知道答案的例子，并做实验来证实你的理解。如果一个结果让你太困惑，早期阶段不要花时间在它上面。

3 - 5

_2

_2，而不是-2，可能会让你困惑。别担心，一会儿就会解释的。

本书中的大多数示例会显示你应输入的内容（缩进三个空格），同时也会显示系统返回的结果。这意味着你可以轻松地通读本入门手册，无需实际操作系统即可查看结果。然而，真正学会的唯一途径，最终还是靠你亲自尝试这些示例，并用自己的想法进行实验。示例采用等宽字体显示，与它们在系统Term窗口中的呈现方式基本一致。部分示例会使用较大的字体，以便你更轻松地阅读并在系统中输入——这通常出现在你可能因对某些词汇不熟悉而输错内容，或者输入错误可能导致令人困惑的结果时。

在做实验的时候，你经常想对你已经输入过的句子做一些细微的改变。有几个捷径可以使这更容易。在Term窗口中，你可以将光标移动到窗口中

的任意行，然后按Enter键，将该行作为窗口底部的新行，以便进行编辑。通过按住Ctrl+Shift并按向上箭头键，可以调出以前的输入行进行编辑，持续按向上箭头直到看到想要处理的行。

大多数小节中的示例都是独立的，但小节的后面部分可能依赖于前面部分中采取的步骤。有些小节依赖于前面几节中采取的步骤，但这应该是相当明显的。

第 8 章

标准配置

本书中的示例假设系统在JQt IDE中启动时运行标准配置文件。这将配置你的系统并提供一些标准实用程序。

在Term窗口中输入CR（大写字母C后跟大写字母R），以快速检查配置文件是否已运行。

```
CR
```

如果运行结果是空行，那么标准配置文件已经运行，你可以跳过本节的其余部分。

相反，如果你看到：

```
CR
```

```
|value error
```

那么你必须更改系统，以便运行标准配置文件。一个正常的J安装总是被配置为运行标准配置。 参考[安装J](#)。

第 9 章

术语

所有的编程语言都与英语有共同之处。在类比含义接近的情况下，J编程语言倾向于使用英语术语，而不是数学和其他编程语言中使用的术语。

像在其他语言中一样，你可以说“代码行”，但在J语言中，倾向于说“句子”。类似地，你可以把+称为“函数”，但在J中通常称为“动词”。

J使用的英语术语有：字母表、单词、句子、短语、动词、名词、副词和连词。

采用这种方法有几个原因。它解决的一个问题是：传统术语在数学和许多编程语言中有大量相关但又微妙不同的用法。例如：函数、子函数、运算符、程序、例程和子例程在不同的编程语言中都以略有不同的方式使用。J没有继承这种混乱，而是采用自己的术语，并在其上下文中精确地定义它们。

使用英语术语可以让你很好地了解术语在J中的一般含义。此外，使用自然语言术语可以鼓励和促进对问题进行英语陈述，并更直接地编写相应的J句子。

我们鼓励使用J术语，但肯定不是强制性的，使用术语“函数”而不是“动词”是完全可以接受的。

第 10 章

字母表

J编程语言的字母表是ASCII字母表，由以下部分组成：

26 lowercase letters (a to z)

26 uppercase letters (A to Z)

0 1 2 3 4 5 6 7 8 9

= < > _

+ * - %

^ \$ ~ |

. : , ;

! / \

[] { }

" ` @ & ?

()

,

有几个字符有时会引起混淆。-（减号）字符不同于_（下划线）字符，并且有三种不同的引号字符：

' quote

" double-quote

` back-quote

如果你尝试一个例子，输入"（双引号）误输入为''（两个单引号），你会感到失望，因为你的结果与书中的不一样。

句号.通常称为点。

第 11 章

单词

单词是字母表中有意义的一组字符。

2.5 + 5

这个句子有三个单词：数字2.5，+和数字5。

用字符组成句子中的单词的规则很简单，目前，常识足以识别句子中的单词。有一些复杂情况在后面的章节中讨论。

第 12 章

句子

句子是由一组单词组成的完整指令。

下面的2加5指令就是一个J语言的句子。

`2 + 5`

与英语句子不同，J语言的句子不以句号或其他标点符号结尾。

相反，J编程语言的句子通常是一个完整的行。

第 13 章

动词

在下面的句子中+是动词（表示动作的单词）。

2 + 5

第 14 章

名词

在下面的句子中，数字2和5都是名词。

$$2 + 5$$

第 15 章

数字

以下这些是数字：0 12 02.5 12.75 0.5 7e6

输入这些数字或其他数字，用动词+ - * %组成简单句子。

2 * 12.75

25.5

一个重要的规则是数字不能以点（.）开头。

0.5

0.5

0.5 + 3

3.5

.5

+--+--+

|.|5|

+--+--+

`.5 + 3`

| `syntax error`

| `.5+3`

显然，`.5`和`0.5`不一样。你现在不需要知道`.5`是什么，但重要的是明白数字不是以点开头的。

`_`（下划线）是无穷大，是一个数字。

第 16 章

负数

负数用_（下划线）表示，而不是-（减号）。

5 + _3

2

_7

_7

_5e_3

_0.005

5 - 9

_4

去掉-和9之间的空白并不会改变含义。

5 -9

_4

-总是动词。这简化了识别句子的规则，因为没有把-直接用在数字左边这种情况。代价就是，既然-总是一个动词，则需要另一个字符_（下划线）来拼写负数。

_用于拼写数字，放在有0 1 2 3 4 5 6 7 8 9，. 以及表示指数符号的e组成的数字前；表示一个负数。

记住：数字前面的 $\cdot_ \cdot$ 是数字的一部分，表示它是负数，而-总是一个动词，不是数字的一部分。

--（两个下划线）表示负无穷，是一个数字。

第 17 章

原语

原语是由系统定义的一个单词。例如，+是一个原语。原语的含义是固定的，不能改变。

原语可以由一个图形字符（如+）拼写的，或者用图形字符加上一个表示变形的字符（点号或冒号）拼写，如+.或者+:

原语也可以由一个或多个字母后跟一个点号或冒号拼写。例如，i.是一个原语，根据它的使用方式的不同，被称为**生成索引**或**求索引**。

第 18 章

名字

原语是由系统定义的单词，而名称是由你定义的单词。原语=.定义一个名字。

5 + v

28

这个句子可以读作v是23。单词=.叫做**系动词**（另一个很好的英语术语）。名字v被定义为数字23，可以用在其他句子中。

与原语不同，名字可以重新定义。

v =. 45

5 + v

50

当句子以系动词开始时，系统不会显示句子的结果。只包含名字的句子将显示名字所定义内容的显示形式。

abc =. 123

abc

123

你可以给任何东西起名字。例如，你可以给动词+起一个名字。

```
plus =. +
```

```
23 plus 45
```

68

`abc =. def`的一般读作：`abc`被定义为`def`。然而，借用其他计算机语言，也常说：`abc`被赋值为`def`或将值`def`赋给`abc`。

第 19 章

注释

原语NB.开始一个注释，该注释一直到行尾结束。

```
2 + 23  NB. this is a comment and is not executed
```

25

第 20 章

错误

句子中的错误将会被报告并停止执行。

```
123 foo 234
```

```
|value error: foo
```

```
| 123      foo 234
```

一条竖线标记了错误报告。如果可以的话，句子会用额外的空格来标记检测到错误的地方。在J9.4版本中，错误消息得到了改进，使调试更加容易。

第 21 章

两可性

每个动词都有两种定义：当它同时有左参数和右参数时使用双子（**dyad**），当它只有右参数时使用单子（**monad**）。这两个定义通常是相关的，比如-的双子定义表示减法，而单子定义表示求负。

5 - 3

2

- 7

_7

双子也被称为动词的二元模式，而单子被称为动词的一元模式。“两可性”一词用于化学意义上的两种价态，表明一个动词既可以与一个参数反应，也可以与两个参数反应。

记住：绝大多数动词都有单子和双子两种定义。

第 22 章

双子

如果一个动词同时有左参数和右参数，则使用动词的二元模式。

双子%（除以）被定义为左参数除以右参数。

5 % 2

2.5

第 23 章

单子

当动词只有一个右参数时，使用动词的一元模式。

单子%（倒数）被定义为1除以右边的参数。

% 2

0.5

单子%和双子%之间的关系很常见，其中单子是带有固定左参数的双子，你会在许多其他原语动词中看到这种关系。

第 24 章

NuVoc – 新词汇表

[NuVoc](#)（菜单Help|Vocabulary）是最新的J参考，是你关于J的最终、权威信息来源。你越早熟悉使用它越好，一个好的开始方法是查找到目前为止介绍的原语。

在JQt IDE中，按F1显示NuVoc，并单击原语跳转到条目。如果你按下F1同时按住Ctrl（Mac的Cmd），你会得到上下文敏感的帮助。如果光标在+处，按Ctrl+F1将跳转到原语+的条目。

在NuVoc中，查看包含+的第6行。这一行的第一项包含：+ 0 Conjugate . Plus 0 0。单词+是动词。它的单子称为求共轭，它的双子称为加法，就像算术中定义的那样。

单子+很有趣。在数学中，复数的共轭是实部相同，虚部相反的数。在实数上求共轭没有作用，结果是参数本身，在复数上它改变虚部的符号。J支持复数，就像直接支持整数或实数一样。复数用j分隔实部和虚部来表示。

```
int =. 23
```

```
+ int
```

```
23
```

```
float =. 23.5
```

+ float

23.5

imagine =. 2j3 *NB. 2 real part, 3 imaginary part*

+ imagine *NB. change sign of imaginary part*

2j_3

许多原语支持复数，NuVoc必须对此进行记录，这意味着在某些描述中存在一些额外的复杂性。如果你的应用程序中需要复数，这是非常棒的。但如果你是一个初学者，不关心复数，那么你必须知道足够的知识，从而能够忽略掉这些，而不会分心或困惑。

让我们看一下wiki中的Plus条目。你可以通过点击Plus链接访问。这将带你到描述+的页面中间。 wiki页面的上半部分是描述Conjugate的信息。Plus的标题行显示它是一个带参数x和y的双子动词。 下面是一个指向Rank 0 0的链接，以及一个指向为什么这很重要的链接。你很快就会知道的。

接着下面是+的简要描述。然后，该页面继续进行一般性讨论和示例。

回到NuVoc，看看+列右边的+.条目，它包含：+. Real/Imaginary . GCD (or)， 其中Real/Imaginary是双子，GCD (or)是单子。我们来看一下定义。

参考页面的上半部分涵盖了+.的Real/Imaginary用法。下半部分覆盖GCD (or)。

二元模式被描述为：GCD (Or)，它有两个非正式的名称，GCD和Or，表明它可以以两种不同的方式使用。 请注意，GCD代表最大公约数（这至少应该唤起数学记忆）。进一步在定义中，你将发现，如果参数是布尔值，则+.是逻辑or函数。 GCD是将or函数的域扩展到非布尔参数的有用扩展。这种原语域的扩展在J中很常见。现在有趣的是，请注意+.有这个较大的域，但也很容易把它限制到布尔参数上。

0 +. 0 *NB. 0 or 0*

0

```

0 +. 1
1
1 +. 0
1
1 +. 1
1

```

在NuVoc中再移动一列，我们可以看到词汇表条目+:包含: Double . Not-Or。该定义对单子和双子都给出了非常简单的定义。

一元模式的情况称为加倍，它的作用正如你所期望的那样。

```

+: 3
6

```

二元模式是实参逻辑或运算的否定。

同样，NuVoc中的一些一般性讨论和示例现在可能超出了你的能力范围。但关键是要知道如何导航并获取相关信息。

?和?.行之后，有一些其他原语行，这些原语是由用点或者冒号作为后缀的名字组成的。在原语表的下面是一个辅助页面列表，它们解释了J的一些概念。它们中的一些在你完成本入门之后值得一看，因为它们提供了你将看到的一些概念的更详细的信息。

第 25 章

第一个检查点

到了这个检查点，你应该明白：

- 如何使用J字典词汇表
- 术语：单词、句子、名词、动词、矛盾词、双子、单子等

通过做以下练习来检查你的理解：

- 查找单子+：*：-：%：的定义
- 试试这些新单子

第 26 章

数值常量

你已经看到了单个数字的使用。也可以有一个数字列表。

```
num =. 5
```

```
nums =. 23 0.5 12.5 7e6 _12 7
```

你将会对数字列表做更多操作，但现在我们只想建立这样一个列表。

第 27 章

字符串

用单引号括起来的字符创建一个字符串。

```
char =. 'Q'
```

```
chars =. 'this is a list of characters'
```

显示字符串时不显示单引号。

```
char
```

```
Q
```

```
chars
```

```
this is a list of characters
```

单引号用于字符串的开始和结束。一对单引号表示引号字符本身在字符串中。

```
'put 2 ''s to get 1 '' in the string'
```

```
put 2 's to get 1 ' in the string
```

不匹配的单引号是错误的。

```
abc =. 'asdf
```

```
|open quote
```

字符串可以为空。

```
abc =. ''
```

字符串也被称为字面值常量或字符常量。

第 28 章

单词构成

单词::可以帮助你找出句子中的单词。单词构成原语以字符串作为其右参数，将其分割成单词，并返回结果，每个单词都在一个盒子中。先不要担心这些盒子是什么，只需注意它们在视觉上的帮助。你将在后面的部分中了解盒子。

```
单词:: '2 + 3'
```

```
+-----+
```

```
|2|+|3|
```

```
+-----+
```

```
单词:: '2.5 + 3e4'
```

```
+---+---+
```

```
|2.5|+|3e4|
```

```
+---+---+
```

```
单词:: 'a =. 1 2 3'
```

```
+---+---+
```



```

|a|=.|1 2 3|

+--+---+-----+

      ;: 'test + 123 NB. this is a comment'

+-----+--+---+-----+

|test|+|123|NB. this is a comment/

+-----+--+---+-----+

      ;: 'def =. ''testing 1 2 3'''

+-----+--+---+-----+

|def|=.|'testing 1 2 3'|

+-----+--+---+-----+

```

请注意，以下都是J单词，每个单词都在自己的盒子里：

2.5

3e4

=.

1 2 3

test

NB. this is a comment

'testing 1 2 3'

你可能会感到惊讶，诸如1 2 3和'testing 1 2 3'之类的常量都是J单

词。 这是很重要的一点，理解这一点在阅读和书写J句子时是必要的。

如果你确实对一个J句子感到困惑（这是可能发生的），你可以做的一件事就是对它应用;:， 以确保你知道构成它的单词。然后你就可以关注这些单词的意思了。

在[NuVoc](#)中查找;:。;:的非正式名称单词构成。

第 29 章

空格

不需要空格来分隔原语和其他单词。另一方面，额外的空格也不会改变意思。

`2+3`

`5`

`2 + 3`

`5`

`2    +    3`

`5`

本书中的大多数示例在所有原语周围都有空格。这使得单个单词显而易见，让你专注于单词的意思，而不需要先弄清哪些是单词。

然而，大多数J程序员，随着他们变得越来越有经验，达到了可以轻松阅读句子中的单词的程度，而额外的空格变成了理解的麻烦和障碍，而不是帮助。你将注意到，在后面的一些示例中，省略了一些不必要的空格。

在某些情况下，空间是必不可少的。

名字之间必须用空格分隔。

```
a=.3
```

```
plus=.
```

```
a plus a
```

```
6
```

```
aplusa
```

```
|value error: aplusa
```

而.（点）和:（冒号）用作单词屈折变化，将紧接在它们前面的单词变成一个新词。当它们用作原语或原语的开头时， 必须在前面有一个空格，这样它们就不会被视为屈折。

```
;: 'a . b'  NB. 3 words
```

```
+--+--+
```

```
|a|.|b|
```

```
+--+--+
```

```
;: 'a .b'  NB. same 3 words
```

```
+--+--+
```

```
|a|.|b|
```

```
+--+--+
```

```
;: 'a. b'  NB. 2 words
```

```
+---+--
```

```
|a.|b|
```

+--+--+

如上所示，. 或者: 如果不是用作屈折，则必须和它们之前的单词之间用空格分开。

数字，例如1e7，可以包含字母。事实上，在J中，有几个字母可以用于拼写数字。而紧跟着数字的字母将被当作数字拼写的一部分。

plus =. +

1plus 3

| ill-formed number

| 1plus 3

| ^

1 plus 3

4

必须用空格将数字与非数字组成部分的字母分开。

第 30 章

优先级

数学传统上认为乘法优先于加法。在数学课上，如果有人问你 $2 + 3 * 4$ 等于多少，你要知道答案是14。

如果只有几个函数和几个级别的优先级(除法|乘法|加法和减法)，这并不太令人困惑。但事情在诸如C这样的语言中会变得笨拙，因为有许多功能和优先级级别。

由于J中有大量的动词，很难定义优先级，更不用说在阅读或编写时试图记住它了。此外，能够命名事物意味着你还必须弄清楚如何处理这个句子：

```
2 plus 3 times 4
```

与传统数学以及大多数其他编程语言不同，J没有动词优先级。你可能需要一点时间先停止乘法，但整体的简化是值得的。

记住：没有动词优先级。

第 31 章

括号

数学和大多数编程语言（包括J语言）都使用括号来控制句子的求值。如果一个句子被完全括起来，那么大多数语言的求值顺序是相同的，并且不受动词优先级或任何其他规则的影响。

$$2 + (3 * 4)$$

14

$$(2 + 3) * 4$$

20

$$10 - (4 - 3)$$

9

$$(10 - 4) - 3$$

3

对这些答案没有任何异议。

第 32 章

求值顺序

如果省略括号，答案是什么？

$$10 - 4 - 3$$

9

J对句子求值为：

$$10 - (4 - 3)$$

9

大多数其他语言会将其求值为：

$$(10 - 4) - 3$$

3

在没有括号的情况下，J从右到左隐式地产生语句。其他语言从左到右产生它们。 一个更长的句子会让这看起来更清晰。

$$10 - 4 - 3 - 1$$

8

10 - (4 - (3 - 1)) *NB. J right-to-left*

8

((10 - 4) - 3) - 1 *NB. others left-to-right*

2

现在考虑一组一元动词。

- - - 4

_4

每个人都知道如何加括号，每个语言都是一样的。

- (- (- 4))

_4

分组是从右向左进行的，在这种情况下，其他语言和J一样。J总是从右向左插入括号，而其他语言对于不同的情况有不同的规则。

J的求值顺序是从右到左。大多数其他语言都是从左到右对二元动词求值，从右到左对一元动词求值；而求值顺序又会被所涉及的动词的相对优先级修改。

对于名词和动词，[J词典E章节](#)的J求值规则是：

执行从右向左进行，但当遇到右括号时，先执行它及其匹配的左括号所包含的段，用其结果替换整个段及括号。

我们应该注意到，虽然[NuVoc](#)是J信息的权威来源，但J字典中有很多有用的信息可能没有进入wiki。

在J中，除了名词和动词之外，还有一些你还没见过的东西，它们会让这个规则变得更加复杂。正是这些额外的类在很大程度上证明了J打破传统 并采用从右到左的求值是合理的。进一步引自[J词典E章节](#)：

这些规则的一个重要后果是，在没有括号的表达式中，任何动词的右参数都是其右侧整个短语的结果。

这是因为没有动词优先级以及从右到左的求值的结果。

没有动词优先级、从右到左的求值以及其他类的规则使总体求值规则变得简单，减少了对括号的需要，并且使有经验的J用户更容易阅读和编写句子。

阅读下面的句子，在你的脑海中对它们求值，并理解无优先级和从右到左求值是如何解释答案的。

$$2 * 4 + 5$$

18

$$2 + 4 * 5$$

22

$$2 - 4 - 5$$

3

$$8 \% 2 + 2$$

2

记住：没有动词优先级和从右到左的求值。